Lucas Ziquinatti - Rafael Lehnhart

DEBUGGERS

A BATALHA CONTRA OS BUGS

🧠 Sobre o Jogo

Debuggers é um jogo de lógica e programação, criado com base na linguagem **Python**.

Ele foi desenvolvido para ajudar jogadores a **raciocinar como programadores**, praticando a construção de algoritmos reais em equipe, de forma lúdica e acessível. A linguagem Python foi escolhida por sua clareza e simplicidade — ideal para quem está começando.

Durante a partida, os jogadores vão interagir com **conceitos reais de programação**, como variáveis, condicionais, loops e operadores. Para tornar a experiência mais dinâmica e amigável, **alguns elementos da linguagem foram adaptados ou simplificados**, como as estruturas FOR, PRINT e as declarações de tipo.

💡 **Ajuda nas cartas**:
A maioria das cartas traz uma breve explicação do elemento representado e um **exemplo prático e real de uso**, ajudando os jogadores a aprender enquanto jogam.

1. 🔍 Resumo do Jogo

Debuggers é um jogo cooperativo de **programação e raciocínio lógico**. Os jogadores assumem o papel de programadores que precisam construir um sistema funcional antes que os Bugs destruam o projeto.
Durante a partida, os jogadores deverão resolver desafios de lógica para construir algoritmos. Se falharem em manter a **estabilidade do sistema**, os Bugs se acumulam — e o projeto será cancelado.

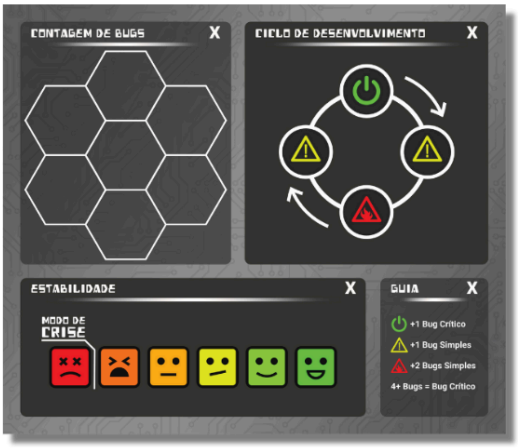
Número de Jogadores

Debuggers foi desenvolvido para **4 jogadores**, oferecendo o equilíbrio ideal entre colaboração, estratégia e distribuição de tarefas. É possível adaptar a experiência para grupos de **2 a 5 jogadores**, mas algumas mecânicas (como o fluxo de cartas e a resolução paralela de desafios) funcionam de forma mais fluida com quatro participantes. Em grupos menores, recomenda-se aumentar a comunicação e dividir as responsabilidades entre os jogadores.

💡 **Jogadores e Programadores**:
Ao jogar **Debuggers**, cada jogador é um programador. Assim, neste Livro de Regras a palavra “**programador**” se refere diretamente a cada jogador.

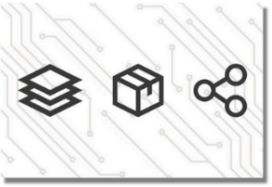
COMPONENTES

1 Tabuleiro



4 Cartas de Desafio

28 Cartas Auxiliares



Cartas exclusivas de cada Desafio. A quantidade varia para cada Desafio.

114 Cartas de Código

27 Cartas de Bug Crítico

4 Cartas de Resposta

1 por Desafio

12 Cartas de Dica

3 por Desafio

Lista de Cartas de Código

Tipos:
- 9x Int
- 9x Float
- 5x Bool
- 4x Str

Aritméticos:
- 9x Soma (+)
- 9x Subtração (-)
- 9x Multiplicação (*)
- 9x Divisão (/)

Ações:
- 9x Print
- 9x Input
- 9x Atribuição (=)

Repetição:
- 5x For
- 4x While

Controle:
- 5x If
- 4x Elif
- 3x Else

Lógicos:
- 3x And
- 3x Or

20 Fichas de Bug Simples

20 Fichas de Parênteses

1 Marcador de Etapa

1 Marcador de Estabilidade

2. 🧰 Preparação do Jogo

Antes de jogar uma partida, siga os próximos passos **em ordem**. A imagem na página 5 apresenta um exemplo visual da preparação.

1. Arrume o Tabuleiro

A. Coloque o **Tabuleiro de Marcadores** no centro da mesa.

B. Posicione o **marcador de Etapa** no espaço verde (símbolo de On-Off) do **Ciclo de Desenvolvimento**.

C. Posicione o **marcador de Estabilidade** no espaço mais à direita ( ).

2. Organize as Fichas

A. Crie o **Banco de Bugs**, com todas as fichas de **Bugs Simples** acessíveis a todos.

B. Crie o **Banco Geral**, com as fichas de Parênteses à disposição dos jogadores.

3. Escolha dos Desafios

A. Os jogadores devem decidir **juntos** quais **Desafios** estarão em jogo.
Cada Desafio possui um nível de dificuldade - **Fácil, Médio ou Difícil** - indicado na própria carta pelos símbolos:

- Fácil

- Médio

- Difícil

Alguns desafios possuem **dependências obrigatórias**, marcadas pelo símbolo de âncora (⚓). Isso significa que eles só podem ser resolvidos **após outro desafio ter sido completado**.

B. Para a primeira partida, recomenda-se:

✔️ “Diga Olá”
✔️ “Média Simples”
✔️ “Contagem Regressiva”

C. Coloque as cartas dos desafios **viradas para cima** no centro da mesa.

Organização e Progresso
• Os Desafios podem ser resolvidos **em qualquer ordem**, desde que **respeitadas as dependências**. • Jogadores são livres para contribuir em **um ou mais desafios simultaneamente**. Não é necessário que cada jogador foque em apenas um desafio. • É possível alternar entre desafios **no mesmo turno**, de acordo com as cartas em mãos e a estratégia do grupo. • Os Desafios funcionam como “tarefas abertas” — podem ser atacados em paralelo, colaborativamente, conforme os recursos disponíveis.

4. Separe e Posicione os Baralhos

A. Remova do jogo as cartas de Dica, Resposta e demais que **não estejam relacionadas** aos desafios escolhidos.

B. Separe as cartas específicas de cada desafio:

- Dica (1 a 3) → viradas para baixo, em ordem.
- Resposta → permanece escondida até a compilação.
- Auxiliares → agrupadas e posicionadas próximas ao respectivo desafio.

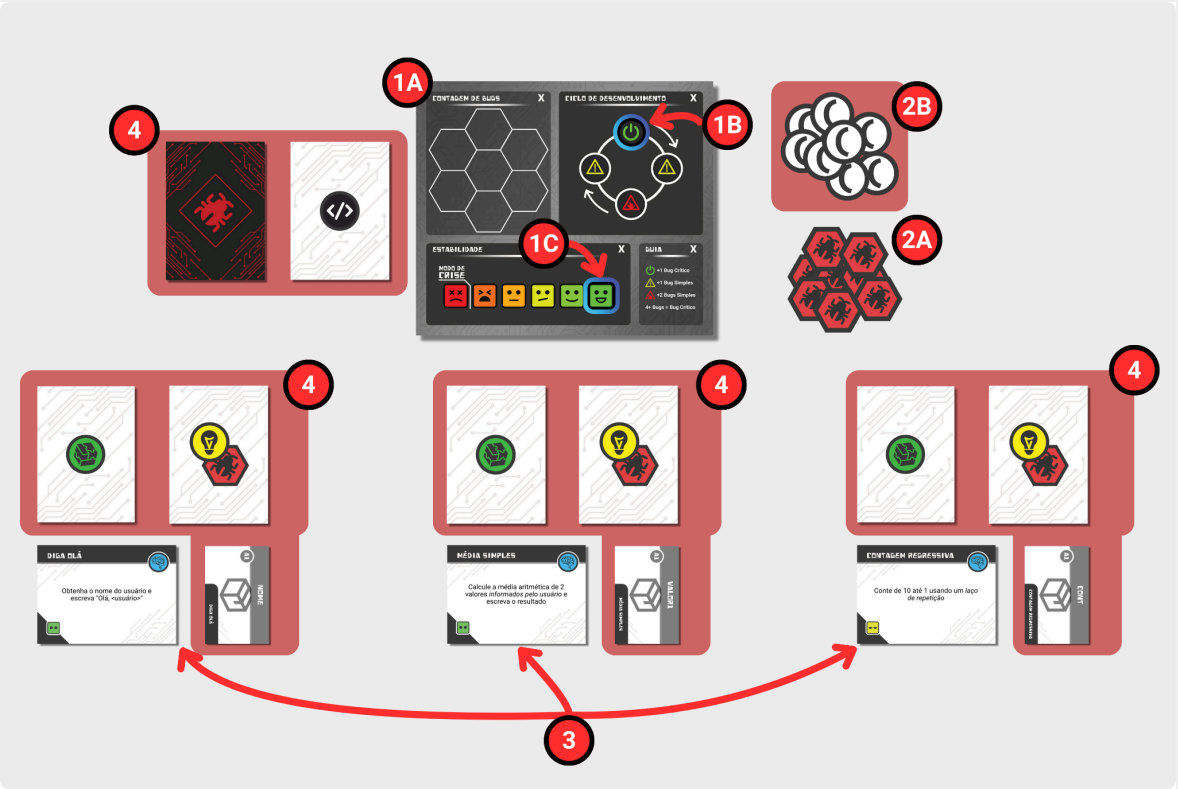
C. Embaralhe os baralhos de:

- **Bugs Críticos**.
- **Código** → distribua **5 cartas para cada jogador**.

5. Abra o Aplicativo

Um jogador deve abrir o aplicativo oficial em seu dispositivo. Ele será utilizado para verificar algoritmos nas **ações de Compilar**.

Diagrama de Preparação da Partida



3. 🎯 Objetivo do Jogo

Todos os programadores jogam **em equipe**, vencem juntos e perdem juntos. Vencerão a partida se conseguirem resolver **todos os desafios**, construindo corretamente os algoritmos solicitados.

⚠️ Mas atenção: **Bugs** surgem durante o desenvolvimento e devem ser corrigidos rapidamente. Se a **Estabilidade** cair demais, o sistema entra em **Modo de Crise** e o projeto pode ser cancelado (consulte “Bugs” na página 9)!

---

4. ▶️ Como Jogar

Estrutura de Rodada

- O primeiro programador é escolhido pelo grupo.
- No início de **cada rodada** (exceto a primeira), todos jogadores compram cartas de Código até possuírem **5 cartas** na mão.
- O primeiro programador realiza **duas ações** e então passa o turno para o programador à sua esquerda, que realiza suas ações, e assim sucessivamente.
- Após todos jogarem, a rodada é finalizada, o **marcador de Etapa avança** e os efeitos do Ciclo de Desenvolvimento se aplicam (consulte “Ciclo de Desenvolvimento”).

Ações Possíveis (explicadas em detalhes mais adiante):

- **Codar**
- **Pesquisar**
- **Revelar Dica**
- **Compilar**
- **Refatorar**
- **Debugar**

⚠️ É permitido realizar ações repetidas, ou seja, um programador pode realizar duas ações iguais, como Codar duas vezes.

Ciclo de Desenvolvimento

O **Ciclo de Desenvolvimento** existe para representar o avanço natural do tempo durante o projeto — e com ele, o surgimento de **Bugs espontâneos**, ou seja, problemas que aparecem independentemente das ações dos jogadores.

Como funciona:

- Ao final de **cada rodada**, o marcador de **Etapa** avança uma casa no Ciclo.
- Isso gera **novos Bugs** e **reduz a Estabilidade do sistema em 1 ponto**.
- A quantidade e tipo de Bugs gerados depende da nova posição do marcador

Ao completar o ciclo (voltando para o espaço verde), um novo **Bug Crítico** é revelado automaticamente (consulte “Bugs” para mais detalhes).

Estabilidade

A **Estabilidade** representa o "estado de saúde" do sistema que os jogadores estão desenvolvendo. Ela tem **6 níveis**, começando no **nível mais à direita** (Estável) e indo até o **modo mais crítico à esquerda**.

Como a Estabilidade é reduzida:

- A cada avanço do marcador de Etapa (fim de rodada), a Estabilidade **diminui 1 ponto**.
- Toda vez que um **Bug Crítico Contínuo** entra em jogo, a Estabilidade é reduzida de acordo com o valor indicado na carta.

Como recuperar Estabilidade:

Sempre que **uma ficha de Bug Simples é removida**, a Estabilidade **aumenta em 1 ponto**. Isso inclui:

- Ação direta de "Debug"
- Remoção de fichas sobre Bugs Críticos Contínuos

Modo de Crise

Se a Estabilidade chegar ao **nível mais à esquerda**, o sistema entra em **Modo de Crise**.

- Se os jogadores permanecerem **2 rodadas consecutivas** nesse estado, **o projeto falha e todos perdem o jogo**.
- Recuperar a Estabilidade durante esse período é a única forma de sair da Crise e evitar a derrota.

5. 🛠️ Ações Detalhadas

📁 Codar

Jogue uma carta de Código da sua mão para um desafio. As cartas devem seguir a **ordem lógica** do algoritmo (consulte “Construindo um algoritmo" na página 8 para mais informações).

⚠️ Para reposicionar cartas já colocadas, use a ação **Refatorar**.

📁 Pesquisar

Descarte 2 cartas da mão → Compre 2 novas do baralho de Código.

📁 Revelar Dica

Revele a próxima carta de Dica do desafio escolhido. Leia em voz alta. Aplique o número de **Bugs Simples** indicados na carta.

📁 Refatorar

Permite reorganizar elementos já jogados (mover, renomear ou remover). Não permite adicionar novas cartas.

📁 Compilar

- No app, selecione o desafio.
- Insira a sequência das cartas jogadas.
- Se for correta: revele a **carta de Resposta** (consulte “Cartas de Resposta” na página 9)
- Se falhar: um **Bug Crítico** é gerado e o Desafio é **bloqueado por uma rodada**.

📁 Debugar

Descarte 1 carta da mão para **remover 1 Bug Simples**. A Estabilidade aumenta em +1. A ficha removida retorna ao banco de Bugs.

⚠️ Só pode ser utilizado quando houver **Bugs Simples** em jogo.

---

6. ✅ Vitória ou Derrota

Vitória

✔️ Compilar com sucesso **todos os desafios**.

Ao compilarem com sucesso o último algoritmo, os jogadores ganham automaticamente.

Derrota

❌ Permanecer **duas rodadas consecutivas** no **Modo de Crise** (Estabilidade zerada).

Sequência para Compilação

Código em Python:	Sequência:
`valor1: float` `valor1 = (2 + 2) / 2`	A1 2 A1 7 (B1 8 B1) 11 B1

7. 📐 Regras Adicionais

Esta seção apresenta todas as demais regras necessárias para se jogar uma partida.

Construindo um Algoritmo

Cada algoritmo é formado por **uma sequência de cartas jogadas na mesa**, organizadas de forma a representar a **lógica de um programa real**, como se estivessem “escrevendo código com blocos”.
As cartas devem ser posicionadas em sequência, respeitando a ordem em que o código seria executado:

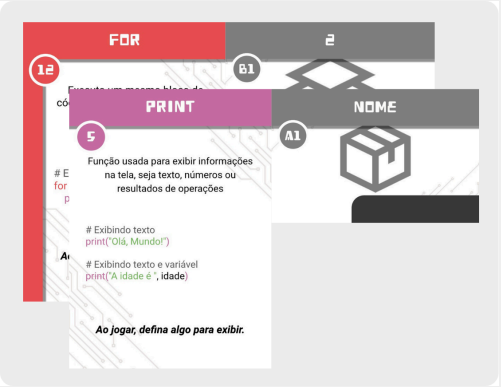
- Monte a sequência da **esquerda para a direita** e de **cima para baixo**.
- Cartas como IF / ELIF / ELSE / FOR / WHILE representam Blocos de Código. Para representar o código interno destes blocos, as cartas devem ser colocadas abaixo mais para a direita, representando o Recuo.

Blocos de Código e Recuo

Código em Python:

```
# Imprime “Fulano” 2 vezes
nome = “Fulano”

for i in range(2):
    print(nome)
```



Tipos de Cartas

Tipo	Uso
Tipo (INT, FLOAT, BOOL e STR)	Declara variáveis (consulte “Type Hint” para uma explicação detalhada)
Aritméticos (+, -, /, *)	Operações entre valores
Controle (IF, ELIF e ELSE)	Iniciam blocos de acordo com as condições estabelecidas
Repetição (WHILE e FOR)	Repetem blocos pelo número de vezes definidas
Lógicos (AND, OR)	Modificam ou combinam condições
Ações (PRINT, INPUT, ATRIBUIÇÃO)	Executam tarefas, dentro ou fora de blocos

Type Hint

Python não é uma linguagem fortemente tipada, ou seja, os tipos são definidos em tempo de execução. Mas como forma de tornar o jogo mais didático e fortalecer boas práticas, as cartas de **Tipo** funcionam como *Type Hints*.

Type Hints são anotações que indicam o tipo esperado de uma variável, parâmetro ou retorno de função, ajudando na leitura do código, na detecção de erros e no uso de ferramentas como editores inteligentes e linters.
Elas **não impedem** o código de rodar com tipos errados, mas servem como uma forma de documentação e verificação estática.

No jogo, as cartas de Tipo têm função semelhante: **informar o tipo esperado de uma variável**, ajudando a organizar o raciocínio lógico do jogador e promovendo boas práticas desde o início.

Cartas de Resposta

Cada desafio possui uma **Carta de Resposta**, que representa o **algoritmo resolvido** daquele desafio — agora encapsulado dentro de uma **função**.
Essa carta é revelada **somente após uma compilação bem-sucedida**, e não pode ser usada antes disso.

O que contém a Carta de Resposta?

A Carta de Resposta mostra o código construído pelos jogadores durante o desafio, mas dentro de uma estrutura de função Python:

```
def nome_da_funcao():
    # Código construído no jogo
    return valor
```

✳ Os jogadores **não precisam jogar as cartas de `def` e `return`** — esses elementos são adicionados automaticamente na carta de resposta, para representar o "encapsulamento" do algoritmo em uma função.

Para que serve?

Após ser revelada, a Carta de Resposta pode ser utilizada em **outros desafios** como uma função reutilizável. Isso significa que os jogadores podem **"chamar" essa função** em outro algoritmo, sem precisar reconstruí-la do zero.
Essa ação representa um conceito importante da programação: **reutilização de código**.

Como usar uma Carta de Resposta em outro desafio?

As Cartas de Resposta devem ser jogadas da mesma maneira que as cartas de PRINT e INPUT.

Caso **retorne um valor**, este deve ser **atribuído** a uma variável.

```
# Função retorna a soma de 2 + 2
soma = soma_dois() # soma recebe o valor 4

# Função para escrever "Olá, Mundo!" na tela
# Não retorna nada
ola_mundo()
```

---

8. 🐞 Bugs

Durante o jogo, **Bugs** representam falhas de lógica, descuidos ou consequências de ações arriscadas. Eles vêm em dois tipos: **Simples** e **Críticos**, e impactam o desenvolvimento do código ao longo das rodadas.

🔸 Bugs Simples

- **O que são?**
  Erros leves que não causam efeitos imediatos, mas **se acumulam ao longo da partida**. No jogo, esses Bugs são representados por Fichas.

- **Como são gerados?**
  - Ao final de cada rodada (Ciclo de Desenvolvimento).
  - Sempre que uma **Dica** é revelada.
  - Por efeitos de **Bugs Críticos**.

➕ Quando um novo Bug Simples for gerado, posicione uma ficha de Bug Simples em um dos espaços de "Contagem de Bugs" do tabuleiro. As fichas nestes espaços representam os Bugs Simples ativos no momento.

➖ Quando um Bug Simples for removido, pela ação de debug ou aparecimento de um Bug Crítico, remova a quantidade de fichas correspondente dos espaços de "Contagem de Bugs" e os devolva ao Banco de Bugs.

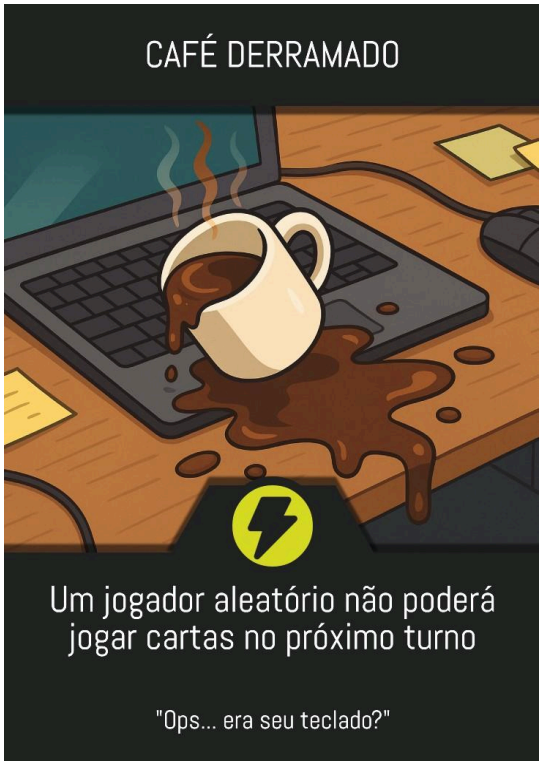
- **Acúmulo perigoso:**
  Se ao final da rodada houver **4 ou mais fichas de Bugs Simples**, 2 são removidas do total e um **Bug Crítico** é revelado imediatamente.

🔺 Bugs Críticos

São falhas sérias no código que causam consequências diretas e impactam a **Estabilidade** do sistema. Existem dois tipos:

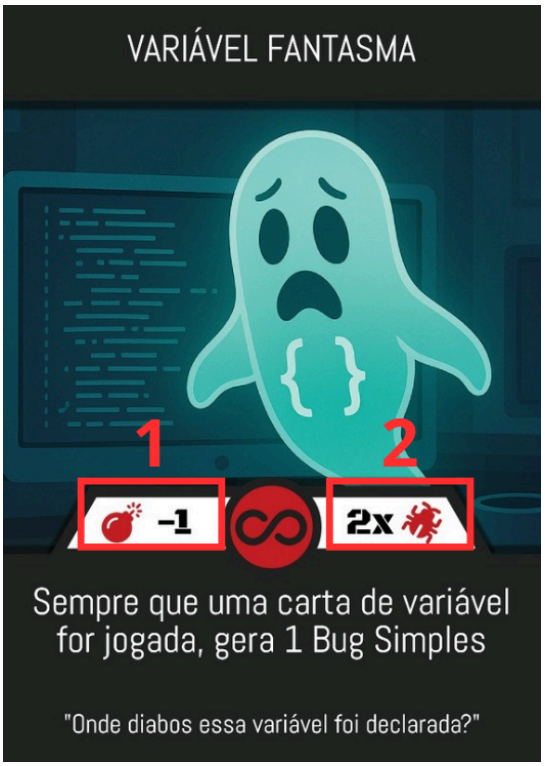
⚡ Bug Flash (Imediato)

- Identificado pelo **ícone de raio**.
- Seu efeito acontece na hora e a carta é **descartada** após ser resolvida.
- **Não reduz Estabilidade.**



♾️ Bug Contínuo (Persistente)

- Identificado pelo **ícone de infinito**.
- **Reduz a Estabilidade** do sistema ao entrar em jogo (1).
- Fica visível ao lado do tabuleiro, com **um número de fichas de Bugs Simples** sobre ele (2). Representa quantos Debugs são necessários para remover o Bug (ex: 2 fichas = 2 ações de Debug necessárias).
- Cada ação de **Debug** remove uma ficha. Ao remover a última, o Bug é resolvido e a carta é descartada.



🧨 Quando um Bug Crítico é revelado?

- Ao fim da rodada, se houver 4 ou mais fichas de Bugs Simples.
- Ao concluir um **Ciclo de Desenvolvimento completo** (marcador retorna ao início - símbolo verde).
- Após uma **Compilação mal sucedida**.
- Como **efeito de outro Bug Crítico**.

💡 Exemplo Prático

- A rodada termina com 5 fichas de Bugs Simples □ **Revela 1 Bug Crítico**.
- O marcador de Etapa completa o ciclo □ **Revela 1 Bug Crítico**.